

Arquivos em Python

Trabalhando com dados externos

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	Por que precisamos de arquivos?	2
2	Interagindo com arquivos	2
3	Criando e abrindo arquivos	2
4	Modos de abertura	2
4.1	Modo leitura — r	2
4.2	Modo escrita — w	2
4.3	Modo append — a	2
4.4	Modo leitura e escrita — r+	3
5	Lendo arquivos	3
5.1	read()	3
5.2	readline()	3
5.3	Percorrendo com for	3
6	Movendo o cursor — seek()	3
7	Escrevendo em arquivos	3
7.1	write()	3
8	Fechando arquivos	3
9	Usando with (recomendado)	4
10	Erros Comuns com Arquivos	5
10.1	1. Esquecer de fechar o arquivo	5
10.2	2. Usar o modo errado	5
10.3	3. Apagar conteúdo sem perceber (modo w)	5
10.4	4. Tentar ler arquivo inexistente	5
10.5	5. Ler o arquivo duas vezes sem seek()	5
10.6	6. Esquecer que write não quebra linha	6
10.7	7. Não usar with (melhor prática)	6
10.8	Resumo	6
11	Lista de Exercícios — Arquivos	7
12	Gabarito — Arquivos	8
13	Mini Projeto Final — Salvando Cadastros em Arquivo	10
13.1	O que o sistema vai fazer?	10
13.2	Código completo do projeto	10
13.3	Como os dados são salvos?	11
13.4	Como o programa lê os dados?	11
13.5	Como o projeto foi organizado	11
13.6	Exemplo de execução	12
13.7	Por que usar with nesse projeto?	12
13.8	Desafios extras	12
13.9	Versão com busca por nome	13

1 Por que precisamos de arquivos?

Até agora, nossos programas só funcionam enquanto estão rodando. Quando o programa termina, os dados são perdidos. Arquivos permitem:

- guardar informações permanentemente
- ler dados depois
- compartilhar informações entre programas

2 Interagindo com arquivos

Python permite interagir com arquivos do computador. Isso significa:

- criar arquivos
- abrir arquivos
- ler conteúdo
- escrever conteúdo

3 Criando e abrindo arquivos

Para trabalhar com arquivos usamos a função `open()`.

```
arquivo = open("dados.txt", "w")
```

Se o arquivo não existir, ele será criado.

4 Modos de abertura

4.1 Modo leitura — r

```
arquivo = open("dados.txt", "r")
```

- Apenas leitura
- Erro se o arquivo não existir

4.2 Modo escrita — w

```
arquivo = open("dados.txt", "w")
```

- Permite escrever
- Apaga o conteúdo anterior

4.3 Modo append — a

```
arquivo = open("dados.txt", "a")
```

- Adiciona conteúdo ao final
- Não apaga o que já existe

4.4 Modo leitura e escrita — r+

```
arquivo = open("dados.txt", "r+")
```

- Permite ler e escrever
- O arquivo precisa existir

5 Lendo arquivos

5.1 read()

```
arquivo = open("dados.txt", "r")
```

```
conteudo = arquivo.read()
```

```
print(conteudo)
```

5.2 readline()

```
linha = arquivo.readline()
```

```
print(linha)
```

5.3 Percorrendo com for

```
arquivo = open("dados.txt", "r")
```

```
for linha in arquivo:  
    print(linha)
```

6 Movendo o cursor — seek()

```
arquivo = open("dados.txt", "r")
```

```
arquivo.read()  
arquivo.seek(0)
```

```
print(arquivo.read())
```

O `seek(0)` volta para o início do arquivo.

7 Escrevendo em arquivos

7.1 write()

```
arquivo = open("dados.txt", "w")
```

```
arquivo.write("Olá, mundo!")
```

```
arquivo.close()
```

8 Fechando arquivos

```
arquivo.close()
```

Sempre feche o arquivo após usar.

9 Usando with (recomendado)

O comando `with` é a forma mais segura de trabalhar com arquivos.

```
with open("dados.txt", "w") as arquivo:  
    arquivo.write("Olá!")
```

Vantagem:

- o arquivo é fechado automaticamente
- código mais limpo

Resumo Final

- `open()` abre arquivos
- modos: `r`, `w`, `a`, `r+`
- `read()`, `readline()` leem conteúdo
- `write()` escreve conteúdo
- `seek()` move o cursor
- `close()` fecha o arquivo
- `with` fecha automaticamente (melhor prática)

10 Erros Comuns com Arquivos

Ao trabalhar com arquivos, alguns erros acontecem com frequência. Conhecer esses erros ajuda muito a evitá-los.

10.1 1. Esquecer de fechar o arquivo

```
arquivo = open("dados.txt", "w")
arquivo.write("Olá")
```

O problema é que o arquivo pode não ser salvo corretamente.

Correto:

```
arquivo = open("dados.txt", "w")
arquivo.write("Olá")
arquivo.close()
```

10.2 2. Usar o modo errado

```
arquivo = open("dados.txt", "r")
arquivo.write("Texto")
```

Erro, pois o modo `r` é apenas leitura.

Correto:

```
arquivo = open("dados.txt", "w")
```

10.3 3. Apagar conteúdo sem perceber (modo `w`)

```
arquivo = open("dados.txt", "w")
```

O modo `w` apaga tudo que já existia no arquivo.

Dica:

Use `a` se quiser adicionar conteúdo sem apagar.

10.4 4. Tentar ler arquivo inexistente

```
arquivo = open("inexistente.txt", "r")
```

Isso gera erro porque o arquivo não existe.

10.5 5. Ler o arquivo duas vezes sem `seek()`

```
arquivo = open("dados.txt", "r")
```

```
print(arquivo.read())
print(arquivo.read())
```

A segunda leitura retorna vazio.

Correto:

```
arquivo.seek(0)
```

10.6 6. Esquecer que write não quebra linha

```
arquivo.write("Linha 1")  
arquivo.write("Linha 2")
```

Resultado:

Linha 1Linha 2

Correto:

```
arquivo.write("Linha 1\n")  
arquivo.write("Linha 2\n")
```

10.7 7. Não usar with (melhor prática)

```
arquivo = open("dados.txt", "w")  
arquivo.write("Texto")  
arquivo.close()
```

Melhor forma:

```
with open("dados.txt", "w") as arquivo:  
    arquivo.write("Texto")
```

10.8 Resumo

- Sempre feche arquivos ou use `with`
- Escolha o modo correto (r, w, a)
- Cuidado com o modo w (apaga tudo)
- Use `seek()` quando necessário
- Use `\n` para quebrar linha

11 Lista de Exercícios — Arquivos

Nível 1 — Básico

1. Crie um arquivo chamado `teste.txt` e escreva "Olá, mundo!".
2. Abra o arquivo e mostre seu conteúdo usando `read()`.
3. Escreva duas linhas diferentes no arquivo usando `write()`.
4. Use `readline()` para ler apenas a primeira linha.

Nível 2 — Manipulação

1. Adicione uma nova linha ao arquivo usando o modo `a`.
2. Leia o arquivo linha por linha usando um `for`.
3. Use `seek()` para reler o conteúdo do arquivo.
4. Crie um programa que peça um nome e salve no arquivo.

Nível 3 — Aplicação

1. Crie um programa que salve vários nomes em um arquivo.
2. Crie um programa que leia esses nomes e mostre todos.
3. Crie um sistema simples que registre nome e idade no arquivo.
4. Mostre todos os dados salvos no arquivo.

12 Gabarito — Arquivos

Nível 1

1.

```
with open("teste.txt", "w") as arquivo:
    arquivo.write("Olá, mundo!")
```

2.

```
with open("teste.txt", "r") as arquivo:
    print(arquivo.read())
```

3.

```
with open("teste.txt", "w") as arquivo:
    arquivo.write("Linha 1\n")
    arquivo.write("Linha 2\n")
```

4.

```
with open("teste.txt", "r") as arquivo:
    print(arquivo.readline())
```

Nível 2

1.

```
with open("teste.txt", "a") as arquivo:
    arquivo.write("Nova linha\n")
```

2.

```
with open("teste.txt", "r") as arquivo:
    for linha in arquivo:
        print(linha)
```

3.

```
with open("teste.txt", "r") as arquivo:
    print(arquivo.read())

    arquivo.seek(0)

    print(arquivo.read())
```

4.

```
nome = input("Digite um nome: ")

with open("teste.txt", "a") as arquivo:
    arquivo.write(nome + "\n")
```

Nível 3

1.

```
with open("nomes.txt", "a") as arquivo:

    for i in range(3):
        nome = input("Digite um nome: ")
        arquivo.write(nome + "\n")
```

2.

```
with open("nomes.txt", "r") as arquivo:

    for linha in arquivo:
        print(linha)
```

3.

```
nome = input("Nome: ")
idade = input("Idade: ")

with open("dados.txt", "a") as arquivo:
    arquivo.write(nome + " - " + idade + "\n")
```

4.

```
with open("dados.txt", "r") as arquivo:

    for linha in arquivo:
        print(linha)
```

13 Mini Projeto Final — Salvando Cadastros em Arquivo

Agora vamos criar um projeto que junta vários conteúdos estudados até aqui. A proposta é montar um pequeno sistema de cadastro que:

- permita cadastrar pessoas
- salve os dados em um arquivo
- leia os dados salvos depois
- mostre os cadastros já existentes

Esse projeto é importante porque mostra uma situação muito comum em programação: **guardar informações para usar mais tarde**.

13.1 O que o sistema vai fazer?

O programa terá um menu com opções:

- 1 → cadastrar uma pessoa
- 2 → mostrar os cadastros salvos
- 3 → sair

Cada cadastro será salvo em um arquivo chamado `cadastros.txt`.

13.2 Código completo do projeto

```
def mostrar_menu():
    print("\n--- MENU ---")
    print("1 - Cadastrar pessoa")
    print("2 - Mostrar cadastros salvos")
    print("3 - Sair")

def cadastrar_pessoa():
    nome = input("Digite o nome: ")
    idade = input("Digite a idade: ")
    cidade = input("Digite a cidade: ")

    with open("cadastros.txt", "a") as arquivo:
        arquivo.write(nome + ";" + idade + ";" + cidade + "\n")

    print("Cadastro salvo com sucesso!")

def mostrar_cadastros():
    try:
        with open("cadastros.txt", "r") as arquivo:
            print("\n--- CADASTROS SALVOS ---")

            for linha in arquivo:
                linha = linha.strip()
                nome, idade, cidade = linha.split(";")

                print("Nome:", nome)
                print("Idade:", idade)
                print("Cidade:", cidade)
                print("-----")

    except FileNotFoundError:
```

```

        print("Nenhum cadastro foi encontrado ainda.")

def main():
    while True:
        mostrar_menu()
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            cadastrar_pessoa()

        elif opcao == "2":
            mostrar_cadastros()

        elif opcao == "3":
            print("Encerrando o programa...")
            break

        else:
            print("Opção inválida. Tente novamente.")

main()

```

13.3 Como os dados são salvos?

Cada linha do arquivo representa uma pessoa cadastrada.
O formato escolhido foi:

```
nome;idade;cidade
```

Exemplo de como o arquivo pode ficar por dentro:

```
Ana;18;Vila Velha
Carlos;20;Vitória
Marina;17;Serra
```

Usamos o ponto e vírgula (;) para separar os dados.

13.4 Como o programa lê os dados?

Quando o programa lê uma linha do arquivo, ele usa:

```
linha.split(";")
```

Isso separa os dados em partes.

Por exemplo:

```
linha = "Ana;18;Vila Velha"
print(linha.split(";"))
```

```
# Saída:
# ['Ana', '18', 'Vila Velha']
```

Assim, conseguimos recuperar nome, idade e cidade separadamente.

13.5 Como o projeto foi organizado

Cada função tem uma responsabilidade específica:

- `mostrar_menu()` → mostra as opções
- `cadastrar_pessoa()` → pede os dados e salva no arquivo
- `mostrar_cadastros()` → lê o arquivo e mostra os dados
- `main()` → controla o funcionamento geral do sistema

13.6 Exemplo de execução

```
--- MENU ---
1 - Cadastrar pessoa
2 - Mostrar cadastros salvos
3 - Sair
Escolha uma opção: 1
Digite o nome: Ana
Digite a idade: 18
Digite a cidade: Vila Velha
Cadastro salvo com sucesso!
```

```
--- MENU ---
1 - Cadastrar pessoa
2 - Mostrar cadastros salvos
3 - Sair
Escolha uma opção: 1
Digite o nome: Carlos
Digite a idade: 20
Digite a cidade: Vitória
Cadastro salvo com sucesso!
```

```
--- MENU ---
1 - Cadastrar pessoa
2 - Mostrar cadastros salvos
3 - Sair
Escolha uma opção: 2
```

```
--- CADASTROS SALVOS ---
Nome: Ana
Idade: 18
Cidade: Vila Velha
-----
Nome: Carlos
Idade: 20
Cidade: Vitória
-----
```

13.7 Por que usar with nesse projeto?

O `with` foi usado porque ele fecha o arquivo automaticamente quando terminamos de usá-lo. Isso é uma boa prática porque:

- evita esquecer de fechar o arquivo
- deixa o código mais limpo
- reduz erros

13.8 Desafios extras

Depois que o sistema estiver funcionando, você pode tentar melhorar o projeto:

- adicionar e-mail no cadastro
- adicionar uma opção para buscar pessoa pelo nome
- mostrar quantos cadastros existem
- salvar também se a pessoa é maior ou menor de idade
- criar uma opção para limpar todos os cadastros

13.9 Versão com busca por nome

Abaixo está uma versão ampliada do sistema, agora com busca.

```
def mostrar_menu():
    print("\n--- MENU ---")
    print("1 - Cadastrar pessoa")
    print("2 - Mostrar todos os cadastros")
    print("3 - Buscar pessoa pelo nome")
    print("4 - Sair")

def cadastrar_pessoa():
    nome = input("Digite o nome: ")
    idade = input("Digite a idade: ")
    cidade = input("Digite a cidade: ")

    with open("cadastros.txt", "a") as arquivo:
        arquivo.write(nome + ";" + idade + ";" + cidade + "\n")

    print("Cadastro salvo com sucesso!")

def mostrar_todos():
    try:
        with open("cadastros.txt", "r") as arquivo:
            print("\n--- TODOS OS CADASTROS ---")

            for linha in arquivo:
                linha = linha.strip()
                nome, idade, cidade = linha.split(";")

                print("Nome:", nome)
                print("Idade:", idade)
                print("Cidade:", cidade)
                print("-----")

    except FileNotFoundError:
        print("Nenhum cadastro encontrado.")

def buscar_pessoa():
    nome_busca = input("Digite o nome da pessoa: ")
    encontrada = False

    try:
        with open("cadastros.txt", "r") as arquivo:
            for linha in arquivo:
                linha = linha.strip()
                nome, idade, cidade = linha.split(";")

                if nome.lower() == nome_busca.lower():
                    print("\n--- CADASTRO ENCONTRADO ---")
                    print("Nome:", nome)
                    print("Idade:", idade)
                    print("Cidade:", cidade)
                    encontrada = True

    except FileNotFoundError:
        print("Nenhum cadastro encontrado.")

    if not encontrada:
        print("Pessoa não encontrada.")
```

```
def main():
    while True:
        mostrar_menu()
        opcao = input("Escolha uma opção: ")

        if opcao == "1":
            cadastrar_pessoa()

        elif opcao == "2":
            mostrar_todos()

        elif opcao == "3":
            buscar_pessoa()

        elif opcao == "4":
            print("Encerrando o programa...")
            break

        else:
            print("Opção inválida. Tente novamente.")

main()
```